

Министерство образования Республики Беларусь

Учреждение высшего образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Контрольная работа
по дисциплине «Микропроцессорные средства и системы»
Тема: «Микроконтроллер MSP430F5529. Описание системы команд
(инструкций) с примерами»

Выполнил:
студент группы 250541 Власов Р.Е.

Проверил:
к.т.н., доцент Селезнёв И.Л.

Минск
2025

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ	3
2 ОПИСАНИЕ МИКРОКОНТРОЛЛЕРА MSP430F5529	3
3 ОПИСАНИЕ СИСТЕМЫ КОМАНД МИКРОКОНТРОЛЛЕРА MSP430F5529	7
3.1 ОПИСАНИЕ РЕГИСТРА ЦПУ И РЕГИСТРА СОСТОЯНИЯ SR.....	7
3.2 ОПИСАНИЕ ДВУХОПЕРАНДНОГО НАБОРА ИНСТРУКЦИЙ	9
3.3 ОПИСАНИЕ ОДНООПЕРАНДНОГО НАБОРА ИНСТРУКЦИЙ	11
3.4 ОПИСАНИЕ НАБОРА ИНСТРУКЦИЙ ВЕТВЛЕНИЙ	13
3.5 РАСШИРЕННЫЕ ИНСТРУКЦИИ СЕМЕЙСТВА MSP430X	14
4 ПРАКТИЧЕСКОЕ ИСПОЛЬЗОВАНИЕ СИСТЕМЫ КОМАНД MSP430F5529	16
5 ЗАКЛЮЧЕНИЕ.....	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24

1 ПОСТАНОВКА ЗАДАЧИ

Изучить устройство и особенности функционирования микроконтроллера MSP430F5529 компании Texas Instruments, а также его систему команд с примерами на языке ассемблера.

2 ОПИСАНИЕ МИКРОКОНТРОЛЛЕРА MSP430F5529

Микроконтроллер Texas Instruments MSP430F5529 (MCU) является частью семейства микроконтроллеров MSP430™ с системой управления и коммуникации сверхмаломощного энергопотребления, состоящего из нескольких устройств с периферийными наборами, предназначенными для различных приложений. Архитектура в сочетании с обширными режимами низкого энергопотребления оптимизирована для достижения длительного срока службы батареи в портативных измерительных приложениях. Микроконтроллер оснащен мощным 16-битным RISC-процессором, 16-битными регистрами, гибкой системой тактирования. Цифровой управляемый генератор (DCO) позволяет устройствам выходить из режимов низкого энергопотребления в активный режим за 3,5 мкс (типично).

Микроконтроллеры MSP430F5529 имеет встроенный USB и PNY, поддерживающий USB 2.0, четыре 16-битных таймера, высокопроизводительный 12-битный аналого-цифровой преобразователь (АЦП), два USCI, аппаратный умножитель, DMA, модуль RTC с возможностями сигнализации и 63 контакта ввода-вывода. Архитектура микроконтроллера представлена на рис.2.1.

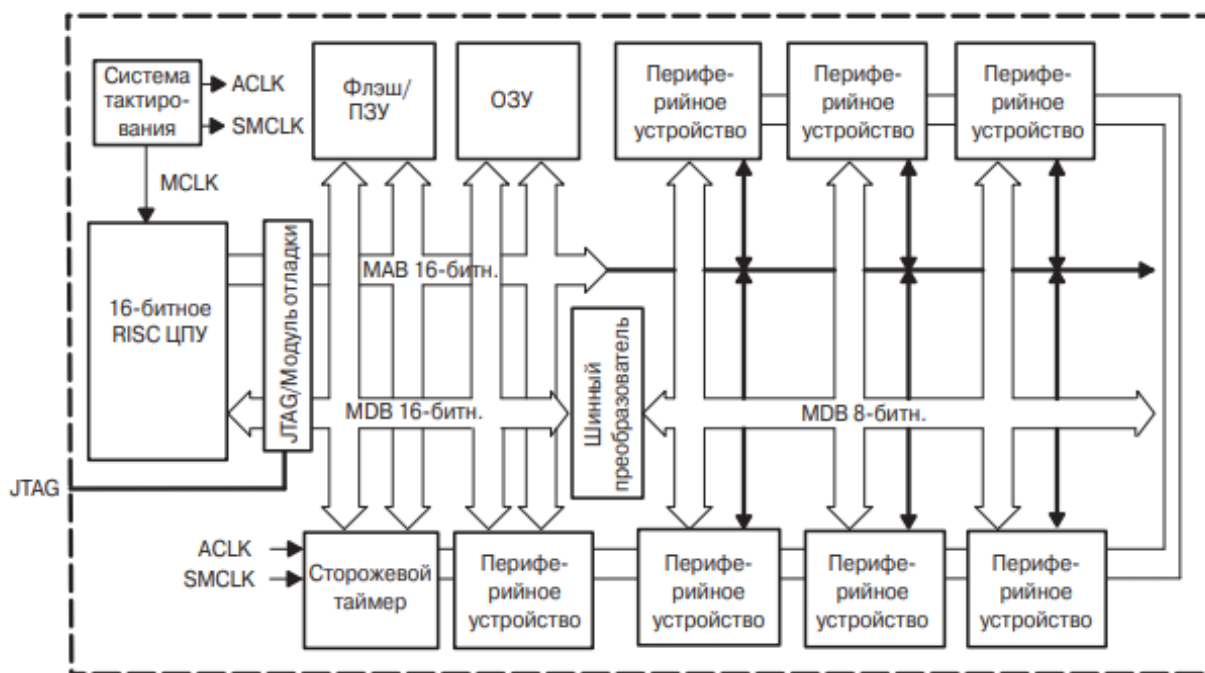


Рисунок 2.1 – Архитектура MSP430F5529

Отличительные характеристики микроконтроллеров семейства MSP430x2xx:

1) Архитектура со сверхнизким потреблением, позволяющая увеличить время работы при питании от батарей:

- ток сохранения содержимого ОЗУ — не более 0.1 мкА;
- ток потребления в режиме часов реального времени — не более 0.8 мкА;
- ток потребления в активном режиме — 250 мкА/MIPS

2) Высокоэффективная аналоговая подсистема, позволяющая выполнять точные измерения:

- таймеры, управляемые компаратором, для измерения сопротивления резистивных элементов.

3) 16-битное RISC ЦПУ:

- большой регистровый файл устраняет ограничения рабочего регистра;
- произведённое по меньшему техпроцессу ядро позволяет снизить потребление и уменьшает стоимость кристалла;
- оптимизировано для современных языков программирования высокого уровня;
- набор команд состоит всего из 27 инструкций; поддерживается 7 режимов адресации;
- векторная система прерываний с расширенными возможностями.

4) Флэш-память с возможностью внутрисхемного программирования позволяет гибко изменять программный код (в том числе, во время эксплуатации), а также производить сохранение данных.

Распиновка выводов для устройства MSP430F5529 в 80 контактном корпусе PN представлена на рис.2.2.

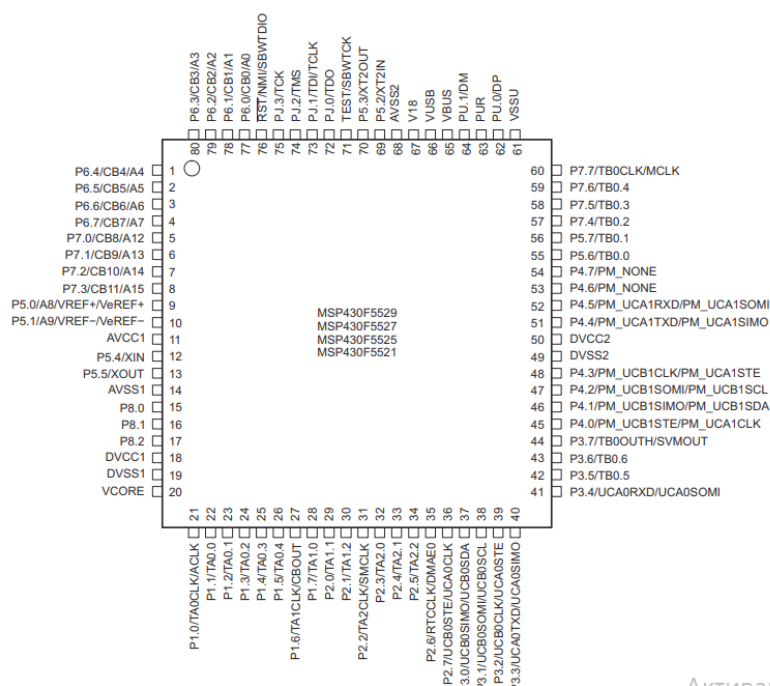


Рисунок 2.2 – Распиновка выводов для устройства MSP430F5529IPN

Система тактирования разработана специально для применения в устройствах с батарейным питанием. Низкочастотный вспомогательный тактовый сигнал ACLK формируется обычным «часовым» кварцем частотой 32 кГц. Сигнал ACLK может использоваться для периодического «пробуждения» часов реального времени, работающих в фоновом режиме. Встроенный высокочастотный генератор с цифровым управлением (DCO) может формировать основной тактовый сигнал (MCLK), используемый ЦПУ и быстродействующими периферийными модулями. Время выхода на режим этого генератора составляет менее 2 мкс при частоте 1 МГц. Решения на базе микроконтроллеров MSP430 эффективно используют высокопроизводительное 16%-битное RISC ЦПУ в течение очень коротких интервалов времени:

- 1) низкочастотный вспомогательный тактовый сигнал используется для реализации режима ожидания со сверхнизким потреблением;
- 2) высокочастотный основной тактовый сигнал используется для эффективной обработки сигналов.

Семейство MSP430 имеет фон-неймановскую архитектуру с единым адресным пространством, которое разделено между регистрами специальных функций (SFR), периферийными устройствами, ОЗУ и флэш-памятью (рис.2.3).

Обращение к исполняемому коду всегда выполняется по чётным адресам. Доступ к данным может осуществляться как побайтно, так и пословно. В настоящее время общий объём адресуемой памяти составляет 128 КБ.

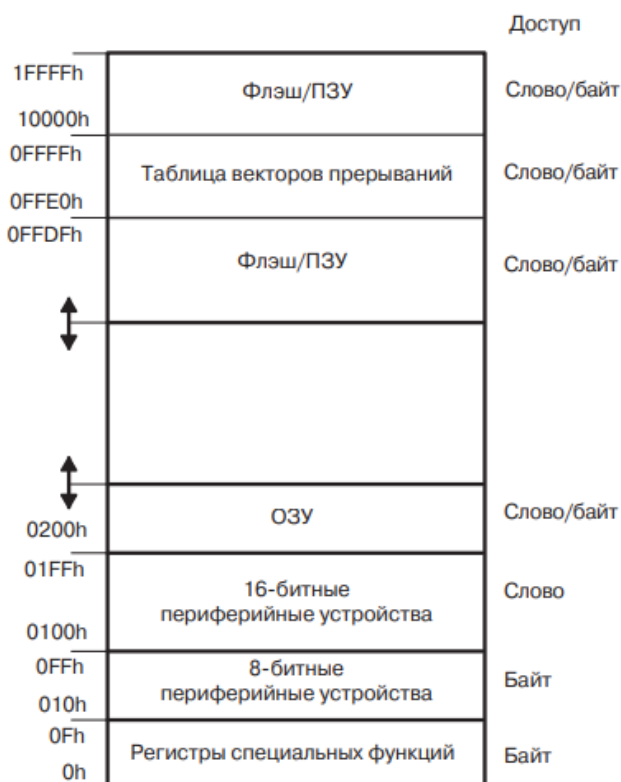


Рисунок 2.3 – Карта памяти

Начальный адрес области флэш/ПЗУ зависит от объёма этой памяти и отличается для разных устройств. Конечный адрес области флэш/ПЗУ всегда равен 0x1FFFF. Флэш-память может использоваться как для хранения кода, так и для хранения данных. Двухбайтные и однобайтные данные (или таблицы данных) могут располагаться во флэш-памяти и использоваться непосредственно оттуда, без предварительного копирования в ОЗУ.

Таблица векторов прерываний занимает верхние 16 слов нижней области памяти размером 64 КБ. При этом вектор прерывания с наивысшим приоритетом располагается в последнем слове области по адресу 0x1FFFF.

Область ОЗУ начинается с адреса 0200h. Конечный адрес области зависит от объёма ОЗУ и меняется от модели к модели. ОЗУ может использоваться как для хранения данных, так и для хранения программного кода.

Регистры периферийных модулей (устройств) располагаются в общем адресном пространстве. Область адресов от 0100h до 01FFh зарезервирована для 16-битных периферийных модулей. Область адресов от 010h до 0FFh зарезервирована для 8-битных периферийных модулей.

Некоторые функции периферийных устройств конфигурируются посредством регистров специальных функций. Эти 8-битные регистры располагаются в младших 16 байт адресного пространства.

ЦПУ MSP430 обладает рядом возможностей, специально предназначенных для поддержки современных методов программирования, таких как вычисляемые переходы, табличные вычисления, а также использование языков высокого уровня, в частности, языка Си.

ЦПУ MSP430 имеет следующие особенности:

- 1) RISC-архитектура, поддерживающая 27 команд и 7 режимов адресации;
- 2) ортогональная архитектура — с каждой из команд может использоваться любой режим адресации;
- 3) полная доступность регистров, включая счётчик команд, регистры состояния и указатель стека;
- 4) одноктактные регистровые операции;
- 5) большой 16-битный регистровый файл, уменьшающий количество обращений к памяти;
- 6) 16-битная шина адреса, обеспечивающая прямой доступ и ветвление во всём диапазоне адресов;
- 7) 16-битная шина данных, позволяющая напрямую оперировать 2-байтными значениями;
- 8) генератор констант формирует шесть наиболее часто используемых значений, уменьшая размер кода;
- 9) прямой обмен данными между ячейками памяти без промежуточной записи в регистр;
- 10) одно- и двухбайтные адресация и форматы команд.

Микроконтроллеры семейства MSP430 широко применяются в устройствах, где критичны низкое энергопотребление и длительное время работы от батареи. Типичными примерами являются:

- портативные измерительные приборы (мультиметры, регистраторы температуры, давления, влажности);
- счётчики электроэнергии, воды, газа и другие приборы учёта;
- медицинские и фитнес-устройства (пульсометры, шагомеры, датчики активности);
- системы удалённого мониторинга и телеметрии;
- датчики и исполнительные устройства в системах «умного дома».

MSP430F5529 удобно использовать в подобных задачах благодаря сочетанию следующих свойств: наличие 12-разрядного АЦП для работы с аналоговыми датчиками, нескольких таймеров для точной временной привязки событий, интерфейсов UART, I²C и SPI для подключения внешних устройств, а также встроенного USB-контроллера, упрощающего подключение к персональному компьютеру. Набор команд, ориентированный на эффективную обработку данных и управление периферией, позволяет реализовывать алгоритмы обработки сигналов и протоколов обмена с минимальным энергопотреблением и небольшим объёмом программного кода.

3 ОПИСАНИЕ СИСТЕМЫ КОМАНД МИКРОКОНТРОЛЛЕРА MSP430F5529

3.1 Описание регистра ЦПУ и регистра состояния SR

Процессор MSP430F5529 использует 16 регистров по 16 бит:

- 1) R0 = PC — счётчик команд (адрес следующей инструкции);
- 2) R1 = SP — указатель стека (адрес области в ОЗУ, куда процессор временно сохраняет данные и адрес возврата);
- 3) R2 = SR/CG1 — регистр состояния и источник используемых констант;
- 4) R3 = CG2 — второй источник констант;
- 5) R4–R15 — обычные рабочие регистры для данных и адресов.

Адреса в системе 20-битные, это означает, что процессор видит линейное адресное пространство больше 64 КБ без страничной логики. Хотя сами операции — 16-битные, доступ к памяти и подпрограммам охватывает весь адресный диапазон. Стек расположен в ОЗУ, при сохранении чего-то в стек указатель SP уменьшается. При входе в прерывание процессор автоматически кладёт на стек SR и полный адрес возврата. Команда RETI

(«return from interrupt») снимает их со стека и корректно возвращает программу в то место, где она прервалась.

SR (Status Register) — панель индикаторов и переключатель питания:

- 1) Z (Zero) — результат равен нулю;
- 2) N (Negative) — установлен старший бит результата;
- 3) C (Carry) — перенос/заём в арифметике; также часто используется как «флаг переноса» при сдвигах;
- 4) V (oVerflow) — арифметическое переполнение для знаковых чисел.

Менять SR необходимо через команды BIS/BIC (установить/сбросить нужные биты) или MOV с маской. Пытаться заполнять в SR произвольные 20-битные значения нельзя — это приводит к непредсказуемому поведению.

Также, необходимо рассмотреть режим адресации — это способ указать процессору, откуда взять данные и куда их положить.

MSP430 поддерживает богатый набор режимов для источника и достаточный — для приёмника:

- 1) регистровый: работа прямо с регистром
MOV R5, R6 — скопировать R5 в R6;
- 2) непосредственный (константа внутри команды).
MOV #1234h, R5 — загрузить число 0x1234 в R5.
- 3) абсолютный (точный адрес в памяти):
MOV R5, &0x2400 — записать R5 по адресу 0x2400.
- 4) символический / PC-относительный (к метке/смещению).
MOV R5, label — то же, что MOV R5, X(PC), удобно внутри модуля.
- 5) индексный X(Rn) (адрес = содержимое Rn + смещение X):
MOV 4(R10), R5 — взять из памяти по адресу R10+4.
- 6) косвенный @Rn (адрес хранится в регистре):
MOV @R10, R5 — взять из памяти по адресу, который лежит в R10.
- 7) косвенный с автоинкрементом @Rn+ (после чтения адрес увеличится):
MOV @R10+, R5 — удобно для чтения массива по указателю.

Для приёмника доступны регистровый, индексный, символический и абсолютный режимы. Если операция обращается к адресу, который не помещается в 16 бит, ассемблер автоматически добавляет к команде дополнительное слово с недостающими старшими битами адреса.

Часто в программе нужны небольшие константы. MSP430 умеет получать их без хранения в самой команде — это делает генератор констант,

привязанный к регистрам R2 и R3. Положительный эффект в том, что команда короче и выполняется быстрее.

В MSP430 разрешены некоторые операции память-память, что экономит регистры. По скорости такой приём не всегда быстрее, чем через промежуточный регистр, но часто удобнее и понятнее.

Базовый набор состоит из 27 инструкций трёх типов:

- 1) двухоперандные;
- 2) однооперандные;
- 3) переходы (ветвления).

3.2 Описание двухоперандного набора инструкций

Двухоперандные инструкции микроконтроллера MSP430F5529 составляют основу его системы команд. Они используются для пересылки данных, выполнения целочисленной арифметики, побитовой обработки и операций сравнения, то есть покрывают большинство типичных операций, встречающихся в прикладных программах.

Назначение и типичные примеры использования двухоперандных инструкций приведены в таблице 3.1.

Таблица 3.1 — Двухоперандный набор инструкций

Инструкция	Назначение (кратко)	Формула/Комментарий	Пример
MOV	Пересылка данных	Source (далее, src) → destination (далее, dst)	MOV.W R5, &P1OUT
ADD	Сложение	$dst = dst + src$	ADD.B #1, R4
ADDC	Сложение с переносом	$dst = dst + src + C$	ADDC.W R6, R7
SUB	Вычитание	$dst = dst - src$	SUB.W #10, R5
SUBC	Вычитание с учётом переноса	$dst = dst - src - (1 - C)$	SUBC.W R6, R7
CMP	Сравнение (без записи результата)	Меняет флаги как при вычитании	CMP.W #0, R5

Продолжение таблицы 3.1

DADD	Десятичное сложение (BCD)	По тетрам, учитывает C	DADD.B R4, R5
BIT	Проверка битов	Меняет флаги по (dst AND src)	BIT.B #BIT3, &P2IN
BIC	Сброс битов	dst = dst AND NOT src	BIC.B #BIT0, &P1OUT
BIS	Установка битов	dst = dst OR src	BIS.B #BIT0, &P1DIR
XOR	Побитовое XOR	dst = dst XOR src	XOR.W R4, R5
AND	Побитовое AND	dst = dst AND src	AND.W #0xFF00, R6

Инструкция MOV является базовой операцией пересылки данных: она используется как для загрузки констант в регистры и копирования значений между регистрами, так и для обмена данными с памятью и периферийными регистрами. В архитектуре MSP430 допускаются даже операции вида «память-память», что позволяет экономить регистры.

Инструкции ADD, ADDC, SUB, SUBC выполняют целочисленную арифметику со знаком и без знака, изменяя флаги Z, N, C, V. Команда ADDC учитывает флаг переноса C и используется при сложении многобайтовых чисел. Аналогично SUBC применяется для многоразрядного вычитания.

Инструкция CMP выполняет вычитание «про себя»: результат не сохраняется, но флаги SR устанавливаются так, как если бы команда SUB dst, src была выполнена. Это позволяет реализовывать конструкции сравнения и ветвления без лишних операций записи.

Команда DADD выполняет десятичное сложение в формате BCD (по 4-битным тетрам), опираясь на флаг C. Она полезна при реализации счётчиков и индикаторов, отображающих данные в человекочитаемом виде (например, на семисегментных или ЖК-индикаторах).

Битовые инструкции BIT, BIC, BIS, XOR, AND обеспечивают компактную и наглядную работу с отдельными битами и полями регистров. BIT позволяет проверить состояние одного или нескольких битов без изменения операнда, BIS и BIC — установить или сбросить нужные биты, а AND и XOR используются для маскирования и инверсии.

Практически все двухоперандные инструкции могут работать как с байтовыми, так и со словными операндами, что определяется суффиксами .B (8 бит) и .W (16 бит) в коде программы. Это даёт возможность обрабатывать как отдельные байты периферийных регистров и данных, так и 16-битные значения без изменения самой логики программ. В сочетании с гибкими режимами адресации и генератором констант двухоперандные инструкции позволяют выполнять большинство операций обработки данных за одну–две машинные команды, а через флаги SR их результаты сразу используются для организации ветвлений и циклов.

3.3 Описание однооперандного набора инструкций

Однооперандные инструкции MSP430F5529 выполняют операции над одним операндом-приёмником. В отличие от двухоперандных команд, здесь явно указывается только один операнд (регистр или ячейка памяти), а источник значения либо совпадает с ним, либо задаётся неявно, например, стек, регистры PC/SR. К однооперандным инструкциям относятся операции сдвига и вращения, работа со стеком, обмен байтов, вызов и возврат из подпрограмм, а также расширение знака. Большая часть этих команд используется для побитовой обработки и масштабирования данных, организации стека и подпрограмм, удобного представления данных в регистре.

Часть однооперандных инструкций изменяет флаги регистра состояния SR, что позволяет сразу после них выполнять условные переходы. Другие работают с адресами и стеком и, как правило, не используются для прямой арифметики над данными.

Назначение и типичные примеры использования однооперандных инструкций приведены в таблице 3.2.

Таблица 3.2 — Однооперандный набор инструкций

Инструкция	Назначение	Пример
RRC	Вращение вправо через C	RRC.B R5
RRA	Арифметический сдвиг вправо	RRA.W R6
PUSH	Поместить значение в стек	PUSH.W R5
SWPB	Обмен старшего и младшего байтов слова	SWPB R7

Продолжение таблицы 3.1

CALL	Вызов подпрограммы (в пределах 64 КБ)	CALL #func
RETI	Возврат из прерывания (восст. SR и PC)	RETI
SXT	Расширение знака 8→16	SXT R4

Инструкции RRC и RRA выполняют побитовые преобразования с участием флага переноса C и при этом обновляют флаги Z и N в соответствии с полученным результатом. Это позволяет использовать их как для арифметических операций (масштабирование, деление на 2), так и для последовательной обработки битов.

Команда PUSH реализует запись слова в стек: указатель стека SP уменьшается на 2 байта, после чего по новому адресу SP записывается значение операнда. Эта инструкция применяется для временного сохранения рабочих регистров, параметров подпрограмм и локальных данных. Обратной по смыслу командой является POP.

Инструкция SWPB меняет местами младший и старший байты слова в регистре, что удобно при работе с байтово-ориентированными протоколами, а также при конвертации форматов данных «младший байт вперёд / старший байт вперёд».

Команда CALL выполняет вызов подпрограммы: в стек сохраняется адрес возврата (текущее значение PC), после чего в PC загружается адрес указанной функции. Возврат из обычной подпрограммы выполняется командой RET, тогда как RETI используется именно для выхода из обработчика прерывания — она восстанавливает не только PC, но и SR из стека, возвращая систему к тому состоянию, которое было до входа в прерывание.

Инструкция SXT (sign extend) выполняет расширение знакового байта до 16-битного слова: младший байт регистра рассматривается как знаковое значение, а старший заполняется копией его старшего бита. Это удобно при работе с 8-битными данными, хранящимися в памяти или периферийных регистрах, когда далее планируется выполнять с ними 16-битные арифметические операции.

В совокупности однооперандные инструкции обеспечивают вспомогательные операции, необходимые для эффективной реализации алгоритмов на уровне машинного кода: от сдвигов и работы с отдельными

байтами до организации стека, подпрограмм и корректной обработки прерываний.

3.4 Описание набора инструкций ветвлений

Инструкции ветвления в архитектуре MSP430F5529 предназначены для изменения последовательного хода выполнения программы в зависимости от состояний флагов регистра состояния SR. На их основе реализуются условные и безусловные переходы, циклы, ветвления по результатам сравнения и различные варианты обработки ошибок.

Все инструкции ветвлений имеют формат `Jxx метка`, где Jxx — мнемоника конкретной инструкции перехода, а метка — адрес (символическое имя) точки в программе, куда должен быть передан контроль. В MSP430 условные переходы анализируют флаги Z, N, C, V регистра SR, при этом сами флаги не изменяются, а их значения наследуются от предыдущей арифметической или логической операции. Большинство инструкций ветвления являются условными, то есть переход выполняется только при выполнении определённого условия над флагами SR. Инструкция JMP является безусловной

Условные переходы используют знаковое смещение длиной 10 бит, измеряемое в словах по 2 байта. Реальный диапазон перехода составляет около ± 1 КБ относительно текущей позиции, чего достаточно для реализации большинства локальных ветвлений и циклов (конструкций типа `if...else`, `while`, `for`) внутри одного фрагмента кода.

Набор основных инструкций ветвления и соответствующие им условия приведены в таблице 3.3.

Таблица 3.3 — Набор инструкций ветвления

Мнемоника	Условие	Комментарий
JEQ / JZ	$Z = 1$	Равно нулю / равны
JNE / JNZ	$Z = 0$	Не равно
JC / JHS	$C = 1$	Перенос / беззнаковое $\geq$
JNC / JLO	$C = 0$	Без переноса / беззнаковое $<$
JN	$N = 1$	Отрицательный результат (знаковый)
JGE	$N == V$	Знаковое $\geq$
JL	$N \neq V$	Знаковое $<$
JMP	—	Безусловный переход

Инструкции JEQ/JZ и JNE/JNZ чаще всего используются после операций сравнения или вычитания для проверки равенства/неравенства операндов. Пара JC/JNS и JNC/JLO ориентирована на беззнаковые сравнения: флаг переноса C интерпретируется как признак результата «меньше/больше либо равно» при работе с беззнаковыми числами.

Инструкции JN, JGE, JL работают со знаковыми числами и используют комбинацию флагов N и V (признак отрицательности и знакового переполнения). Это позволяет корректно реализовывать ветвления для сравнений типа «меньше нуля», «меньше/больше по модулю» при работе со знаковыми 16-битными величинами.

Инструкция JMP выполняет безусловный переход и часто используется для организации бесконечных циклов, переходов к общим участкам кода или реализации таблиц переходов (в комбинации с косвенной адресацией через регистр).

3.5 Расширенные инструкции семейства MSP430X

Классический набор команд MSP430 исторически рассчитан на работу в пределах нижних 64 КБ адресного пространства. Для микроконтроллера MSP430F5529, в котором объём флэш-памяти достигает 128 КБ, этого уже недостаточно: требуется поддержка 20-битных адресов и удобные средства работы с дальними подпрограммами и данными. Эти возможности обеспечиваются расширениями ядра MSP430X.

Основные элементы расширения следующие:

1) CALLA / RETA — команды дальнего вызова и возврата, работающие с полными 20-битными адресами. Подпрограммы могут располагаться в любой точке адресного пространства, а не только в нижних 64 КБ;

2) MOVA — операции переноса 20-битных адресов (указателей). Используются для загрузки и пересылки адресов, в том числе для указателей на «верхние» области памяти;

3) PUSHM / POPM — групповые операции со стеком, позволяющие сохранить или восстановить сразу несколько регистров одним вызовом. Это делает прологи и эпилоги подпрограмм более компактными и быстрыми;

4) укороченные “address-instructions” — специальные варианты некоторых инструкций с ограниченным набором режимов адресации, что позволяет уменьшить длину машинного кода и ускорить его выполнение.

Таблица 4.4 — Расширенные инструкции MSP430X/CPUXV2

Инструкция	Назначение (кратко)	Пример
MOVA	Перенос 20-битных адресов (указателей)	MOVA #far_label, R10
CALLA	Дальний вызов (полный 20-битный адрес)	CALLA #func_far
RETA	Дальний возврат из подпрограммы	RETA
PUSHM	Сохранение группы регистров в стек	PUSHM.W #4, R4 ; R4..R7
POPM	Восстановление группы регистров из стека	POPM.W #4, R4 ; R4..R7

Когда требуется обратиться к константе или области памяти с адресом шире 16 бит, соответствующая инструкция получает дополнительное слово с недостающими старшими битами адреса. Это немного увеличивает размер кода и время исполнения отдельных команд, но обеспечивает полную свободу работы во всём 20-битном адресном пространстве.

Скорость выполнения программы определяется не только набором инструкций, но и тем, как именно код обращается к данным. Регистры обрабатываются быстрее, чем память: операции вида «регистр-регистр» обычно являются самыми быстрыми, тогда как любое обращение к памяти добавляет дополнительные машинные циклы. При использовании адресов, выходящих за пределы 16-битного диапазона, к инструкции добавляется дополнительное слово, из-за чего код становится больше и выполняется немного медленнее.

Отдельно стоит учитывать работу переходов. Условные инструкции типа Jxx выгодно держать «короткими»: они работают в пределах примерно ± 1 КБ вокруг текущей позиции и выполняются быстро. Для дальних переходов, выходящих за этот диапазон, применяют команду JMP или переход через регистр.

Использование небольших констант без дополнительного слова позволяет получить более компактный код и уменьшить число тактов, особенно при обращении к ним через генератор констант. Операции вида «память → память» удобны и позволяют экономить регистры, однако требуют и чтения, и записи в память; в ряде случаев реализация той же логики через временный регистр оказывается более быстрой.

Общее практическое правило заключается в том, что при разработке программ для MSP430F5529 следует по возможности выполнять вычисления в регистрах, использовать генератор констант и выбирать такие режимы адресации, которые минимизируют количество обращений к памяти и длину машинного кода. Расширения MSP430X при этом снимают ограничения на расположение кода и данных и позволяют масштабировать программы на более крупные конфигурации памяти без изменения базовой архитектуры.

4 ПРАКТИЧЕСКОЕ ИСПОЛЬЗОВАНИЕ СИСТЕМЫ КОМАНД MSP430F5529

Работа на ассемблере под MSP430F5529 обычно ведётся в Code Composer Studio (CCS) или через набор msp430-elf с использованием файла заголовков устройства. Типовой каркас включает остановку watchdog-таймера, настройку портов и, при необходимости, тактирования.

Пример начала работы с проектом:

```
001      .cdecls C,LIST,"msp430f5529.h"
002      .text
003 _start:
004      MOV.W    #WDTPW|WDTHOLD, &WDTCTL ; остановить watchdog
005      ; ... дальнейшая инициализация ...
```

Команда MOV и ортогональные режимы адресации позволяют свободно перемещать данные между регистрами и памятью. Поддерживаются регистровый, непосредственный, абсолютный (&ADDR), символический (к метке), индексный X (Rn) , косвенный @Rn и @Rn+.

Инициализация бита порта (работа по байтам, чтобы не затронуть соседние биты):

```
001      .cdecls C,LIST,"msp430f5529.h"
002      .text
003      BIS.B    #BIT0, &P1DIR; P1.0 -> выход (LED)
004      BIC.B    #BIT0, &P1OUT; LED = 0 (выкл)
```

Копирование массива:

```
005      MOV.W    #src, R12
006      MOV.W    #dst, R13
007      MOV.W    #16, R14
008 copy_lp:
009      MOV.B     @R12+, 0(R13) читаем по указателю, пишем по индексу
```



```

010      INC      R13
011      DEC      R14
012      JNZ      copy_lp
013      .bss      src,16
014      .bss      dst,16

```

Стандартные сложение/вычитание — ADD/SUB; варианты с переносом — ADDC/SUBC. Сравнение выполняет CMP: оно меняет только флаги SR (Z/N/C/V), не трогая операнды. Для BCD-счётчиков есть DADD — десятичное сложение по полубайтам.

Инкремент BCD-счётчика в R5 значением из R4:

```

001 .cdecls C,LIST,"msp430f5529.h"
002 BIS.B CLRC; C=0 для корректного DADD
003 DADD R4, R5;
004 BCD: R5 := R5 + R4; Флаги SR отражают BCD-результат

```

Далее рассмотрены битовые операции и проверки (BIT/BIS/BIC/XOR) .

Быстрая работа с отдельными битами — сильная сторона MSP430. BIT проверяет биты по маске, BIS/BIC устанавливают и сбрасывают, XOR «щёлкает» бит.

Мигание LED из прерывания таймера и уход в LPM0:

```

001      .cdecls C,LIST,"msp430f5529.h"
002      .text
003      BIS.B    #BIT0, &P1DIR ; P1.0 — выход
004      BIS.B    #BIT1, &P1REN ; подтяжка на P1.1
005      BIS.B    #BIT1, &P1OUT; подтяжка вверх
006 btn_poll:
007      BIT.B    #BIT1, &P1IN ; Z=1 => кнопка нажата (лог.0)
008      JNZ      btn_poll
009      XOR.B    #BIT0, &P1OUT; переключить LED
010      MOV.W    #50000, R15; простая задержка антидребезга
011 dbnc:      DEC      R15
012      JNZ      dbnc
013      JMP      btn_poll

```

Условные переходы Jxx работают в пределах $\sim \pm 1$ КБ вокруг текущего места и не меняют флаги. Для дальнего безусловного перехода удобно использовать псевдоинструкцию BR label (ассемблер подставит нужный вариант, в том числе с расширением адреса).

Далее рассмотрим стек, подпрограммы, прерывания и режимы пониженного потребления

Стек (SP) используется для адресов возврата и сохранения регистров. Для больших проектов полезны PUSHM/POPM (сохранить/восстановить группу регистров). Вызовы в «верхнюю» память делаются через CALLA, возврат — RETA. Вход/выход из LPM осуществляется через биты SR.

Пример мигания светодиодом из прерывания таймера с уходом в LPM0:

```

001      .cdecls C,LIST,"msp430f5529.h"
002      .text
003 RESET:
004      MOV.W   #WDTPW|WDTHOLD, &WDTCTL
005      BIS.B   #BIT0, &P1DIR; P1.0 — выход (LED)
006      ; TA0: ACLK, up mode, прерывание CCR0 ≈0.5 с при 32.768 кГц
007      MOV.W   #TASSEL__ACLK|MC__UP|ID__1, &TA0CTL
008      MOV.W   #16384, &TA0CCR0
009      BIS.W   #CCIE, &TA0CCTL0
010      ; Разрешение прерываний и сон до таймера
011      BIS.W   #GIE, SR
012      BIS.W   #LPM0, SR
013 ; --- Обработчик прерывания таймера ---
014 TimerA0_ISR:
015      XOR.B   #BIT0, &P1OUT ; мигнуть LED
016      RETI
017 ; --- Привязка вектора сброса к RESET ---
018      .sect   ".reset"
019      .short  RESET
020
021 ;   .sect ".intXX"
022 ;   .short TimerA0_ISR
023 ; где .intXX — правильная секция для TIMER0_A0_VECTOR в
    lnk_msp430f5529.cmd

```

Когда код и данные находятся выше 64 КБ, применяют расширенные инструкции CPUXV2:

- MOVA для загрузки 20-битных адресов в регистры-указатели;
- CALLA/RETA для вызова/возврата по полным адресам.

Пример кода:

```

001      MOVA    #far_table, R10 ; загрузка 20-битного адреса
002      CALLA   #far_func; дальний вызов
003      ; ...
004 far_func:
005      ; тело подпрограммы
006      RETA    ;дальний возврат

```

Далее, представлен полный код программы, предназначенный для демонстрации работы микроконтроллера MSP430F5529 с системными командами с использованием встроенного светодиода и кнопки, а также для реализации простого секундомера. После сброса контроллера останавливается сторожевой таймер, настраивается вывод P1.0 как выход для управления светодиодом, а вывод P1.1 — как вход с подтяжкой вверх для подключения кнопки, работающей по схеме «активный ноль». В регистре R5 ведётся счётчик секунд в формате BCD, который последовательно увеличивается от 00 до 59, после чего обнуляется. Основной цикл программы периодически опрашивает состояние кнопки: при её нажатии состояние светодиода переключается (если был выключен — загорается, и наоборот) с небольшой задержкой для подавлениядребезга контактов. Независимо от нажатий кнопки программа каждую «секунду» (задержка реализована с помощью программного цикла) увеличивает BCD-счётчик, подаёт короткий световой «тик» на светодиоде и затем делает паузу, имитирующую секундный интервал. Таким образом, код одновременно реализует элементарный секундомер на программных задержках и показывает пример работы с портами ввода-вывода, внутренними подтяжками, BCD-счётом и обработкой кнопки.

```

001      .cdecls C,LIST,"msp430f5529.h"
002      .text

003      .retain
004      .retainrefs
005
006 //ТОЧКА ВХОДА
007 RESET:
008      MOV.W    #WDTPW|WDTHOLD, &WDTCTL          ; Остановить watchdog
              (сторожевой таймер)
009      ; WDTCTL — регистр управления WDT.
010      ; Чтобы писать в него, нужно указать "пароль" WDTPW в старших
              битах.
011      ; Бит WDTHOLD = 1 => таймер останавливается.
012      ; Иначе WDT будет периодически сбрасывать МК.
013
014      BIS.B    #BIT0, &P1DIR ; Настроить P1.0 как выход (LED)
015      ; P1DIR — направление порта 1.
016      ; BIT0 соответствует P1.0.
017      ; BIS.B устанавливает указанный бит в 1, не трогая остальные.
018      ; 1 в DIR => вывод работает как выход.
019

```

```

020      BIC.B    #BIT0, &P1OUT ; P1.0 = 0 (LED выключен)
021      ; P1OUT — регистр состояния вывода/подтяжек.
022      ; BIC.B сбрасывает указанный бит (делает 0).
023      ; На LaunchPad светодиод на P1.0 обычно активен высоким
    уровнем,
024      ; поэтому 0 на выводе — LED погашен.
025
026      BIS.B    #BIT1, &P1REN ; Включить резистор на P1.1
027      ; P1REN — enable для подтяжки на порту 1.
028      ; BIT1 -> P1.1.
029      ; 1 в REN включает внутренний резистор (подтяжка к VCC или
    GND) .
030
031      BIS.B    #BIT1, &P1OUT ; Подтяжка вверх на P1.1
032      ; Когда REN=1, значение в P1OUT определяет направление
    подтяжки:
033      ; P1OUT бит = 1 -> подтяжка к VCC (pull-up),
034      ; P1OUT бит = 0 -> подтяжка к GND (pull-down).
035      ; Здесь P1.1 тянется вверх -> кнопка становится "активный 0"
036      ; (при нажатии вход замыкается на землю).
037
038      CLR.W    R5 ; Обнулить BCD-счётчик секунд
039      ; R5 используется как BCD-переменная, хранящая секунды в
    формате 00..59.
040
041 ; Основной цикл программы
042 main_loop:
043      ; --- Обработка кнопки (активный 0 на P1.1) ---
044
045      BIT.B    #BIT1, &P1IN ; Тестируем состояние входа P1.1
046      ; P1IN — регистр входных значений порта 1.
047      ; BIT.B выполняет побитовое AND с константой, результат не
    сохраняет, но выставляет флаги (Zero, Negative и т.д.) в SR.
048      ; Если P1.1 = 1 (кнопка не нажата), результат ≠ 0, Z = 0.
049      ; Если P1.1 = 0 (кнопка нажата), результат = 0, Z = 1.
050
051      JNZ      no_btn ; Если Z=0 (результат ≠ 0) -> кнопка НЕ нажата
052      ; JNZ (Jump if Not Zero) переходит, если Z-флаг = 0.
053      ; То есть при "кнопка отпущена" сразу переходим к метке
    no_btn.
054
055      XOR.B    #BIT0, &P1OUT ; Кнопка нажата -> переключить LED
056      ; XOR по биту P1.0: если был 0 — станет 1, если был 1 —
    станет 0.
057      ; Таким образом, каждый нажим кнопки меняет состояние
    светодиода.
058
059      MOV.W    #50000, R15; Задержка для антидребезга (debounce)

```

```

060          ; При нажатии механической кнопки контакт "дребезжит".
061          ; Этот цикл задержки позволяет дождаться стабилизации.
062
063 deb:
064          DEC      R15 ; Уменьшаем счётчик задержки
065          JNZ      deb ; Пока R15 ≠ 0, крутится в цикле
066          ; В итоге получаем грубую задержку, зависящую от частоты
          тактирования.
067
068 ; Выходим сюда либо если кнопка не нажата, либо после обработки
          нажатия.
069 no_btn:
070          ; BCD-секундомер: R5 := R5 + 1 (диапазон 00..59)
071
072          MOV.W    #1, R4 ; В R4 кладём значение 1 (BCD-единица)
073          ; R4 здесь используется как "прибавляемое" число для BCD-
          счёта.
074
075          CLRC ; Очистить флаг переноса (C = 0)
076          ; DADD (Decimal ADD) использует флаг переноса C как
          "десятичный перенос".
077          ; Чтобы не было "хвостов" от предыдущих операций, его нужно
          сбросить.
078
079          DADD     R4, R5 ; R5 = R5 + R4 в BCD-формате
080          ; DADD — десятичное сложение (BCD):
081          ; - каждые 4 бита рассматриваются как десятичная цифра
          0..9.
082          ; - если в результате получается > 9, автоматически
          прибавляется 6,
083          ; - корректируя число обратно в диапазон 0..9 и устанавливая
          перенос на следующий ниббл.
084          ; В итоге R5 хранит счётчик секунд: 00, 01, 02, ..., 59.
085
086          CMP.W    #0x60, R5 ; Сравнить R5 с 0x60
          (BCD "60")
087          ; 0x60 в BCD — это "60" (6 десятков, 0 единиц).
088          ; После увеличения с 59 на 60 счётчик должен сброситься на
          00.
089
090          JL      tick ; Если R5 < 0x60 -> ещё в диапазоне 00..59
091          ; JL (Jump if Less) — переход, если результат сравнения
          меньше (signed).
092          ; Пока R5 < 0x60, счётчик корректный, ничего делать не нужно.
093
094          CLR.W    R5 ; Иначе R5 >= 0x60 ->
          сбрасываем в 00

```

```

095          ; Таким образом, после "59" следующей секундой будет снова
          "00".
096
097 ; "Тик" светодиода + задержка на 1 секунду
098
099 tick:
100          ; кратковременный вспышка LED, показывающая "тик"
          секундомера
101
102          XOR.B    #BIT0, &P1OUT ; Включить/выключить LED на короткий
          тик
103          MOV.W    #40000, R15 ; Короткая задержка для "тика"
104 d1:      DEC      R15
105          JNZ      d1
106          ; Этот цикл задаёт длительность вспышки.
107
108          XOR.B    #BIT0, &P1OUT ; Вернуть LED в предыдущее состояние
109          ; Теперь светодиод снова в том состоянии, которое было до
          тика.
110
111          ; --- Основная "секундная" задержка ---
112          MOV.W    #60000, R15 ;
113 d2:      DEC      R15
114          JNZ      d2
115          ; После этого времени считаем, что прошла "одна секунда".
116          ; За это время мы один раз увеличили BCD-счётчик и мигнули
          LED.
117
118          JMP      main_loop ; Вернуться в начало основного цикла
119          ; Цикл повторяется бесконечно:
120          ; - опрос кнопки
121          ; - при нажатии — смена LED
122          ; - инкремент BCD-секундомера (00..59)
123          ; - "тик" LED и секундная задержка
124
125 ; Пример выделения памяти в .bss (RAM)
126          .bss     src,16
127          ; В секции .bss выделяется 16 байт под меткой src
          (инициализация нулями).
128          ; Здесь переменная src находится в RAM и по умолчанию
          заполнена 0.
129
130          .bss     dst,16
131          ; Аналогично, ещё 16 байт под меткой dst.
132          ; В этом конкретном примере они не используются — просто
          демонстрация .bss.
133
134 ; Вектор сброса -> адрес метки RESET

```

```

135         .sect    ".reset"
136         ; Переходим в секцию, где лежит вектор сброса.
137         ; Для MSP430 вектор сброса — это 16-битное слово в конце
        памяти, которое указывает адрес, с которого стартует код после reset.
138
139         .short    RESET
140         ; В вектор сброса записываем адрес метки RESET.
141         ; При включении питания или сбросе МК аппаратно подхватит
        этот адрес и начнёт выполнение программы с него.

```

5 ЗАКЛЮЧЕНИЕ

В данной работе рассмотрены микроконтроллер MSP430F5529 и его система команд, в том числе набор из 27 инструкций с ортогональными режимами адресации и генератором констант, что упрощает разработку.

Набор команд MSP430F5529 компактный, но хорошо сбалансирован. Ортогональные режимы адресации и генератор констант позволяют писать короткий и понятный код; битовые операции делают работу с периферией естественной; управление стеком упрощает энергосбережение.

Также, рассмотрены расширенные инструкции MSP430X, которые снимают ограничение нижних 64 КБ и масштабируются под большие проекты. На примерах продемонстрированы типовые приёмы: битовые операции с портами, ветвления Jxx, BCD-арифметика DADD, прерывания и выход в LPM (через биты SR), подтверждающие, что на этой основе можно строить производительные, надёжные и энергоэффективные программы.

Отдельное внимание уделено программе на языке ассемблера, реализующей простой секундомер с использованием встроенного светодиода и кнопки. Практические примеры показывают типовые приёмы: от простых циклов и проверок флагов до прерываний с выходом в LPM. В результате разработчик получает ясные шаблоны для быстрого и надёжного кода, а сама архитектура способствует работе в регистрах и аккуратное обращение к памяти.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Документация на микроконтроллер MSP430F5529 [Электронный ресурс]. – Режим доступа: <https://www.ti.com/lit/gpn/msp430f5529> – Дата доступа: 10.10.2025.
- 2) MSP430x5xx and MSP430x6xx Family User's Guide [Электронный ресурс]. Режим доступа: <https://www.ti.com/lit/ug/slau208q/slau208q.pdf> – Дата доступа: 10.10.2025.
- 3) MSP430F5529 LaunchPad Development Kit (MSP-EXP430F5529LP) User's Guide [Электронный ресурс]. – Режим доступа: <https://www.ti.com/lit/ug/slau533d/slau533d.pdf> – Дата доступа: 10.10.2025.
- 4) Code Composer Studio™ Integrated Development Environment. Installation Instructions and User's Guide [Электронный ресурс]. – Режим доступа: https://software-dl.ti.com/ccs/esd/documents/users_guide_ccs_20.0.0/index_overview.html – Дата доступа: 10.10.2025.
- 5) Семенов Б. Ю. Микроконтроллеры MSP430: первое знакомство [Электронный ресурс]. – Режим доступа: <https://e.lanbook.com/book/13734> – Дата доступа: 10.10.2025.
- 6) Знакомство с микроконтроллером серии MSP430 [Электронный ресурс] : методические указания / сост. Е. И. Шкелёв, В. А. Калинин, В. В. Пархачёв. – Режим доступа: https://www.unn.ru/books/met_files/Znakomstvo_MSP430.pdf – Дата доступа: 10.10.2025.
- 7) Программирование микроконтроллера серии MSP430 [Электронный ресурс] : методические указания / сост. Е. И. Шкелёв, В. А. Калинин, В. В. Пархачёв. – Режим доступа: https://www.unn.ru/books/met_files/first_steps_MSP430.pdf – Дата доступа: 10.10.2025.
- 8) Кратько А. Программирование микроконтроллеров семейства MSP430. Часть I. Аппаратные средства [Электронный ресурс] // Компоненты и технологии. – 2006. – № 4. – Режим доступа: <https://kit-e.ru/programmirovanie-mikrokontrollerov-semejstva-msp430-chast-i-apparatnye-sredstva/> – Дата доступа: 10.10.2025.
- 9) Гук И. Краткий обзор микроконтроллеров семейства MSP430 компании Texas Instruments [Электронный ресурс] // Компоненты и технологии. – 2006. – № 6. – Режим доступа: <https://kit-e.ru/kratkij-obzor-mikrokontrollerov-semejstva-msp430-kompanii-texas-instruments/> – Дата доступа: 10.10.2025.